

HLD & LLD for Real-Time E-Commerce App (with draw.io Guidance)

1. HLD (High-Level Design) Overview

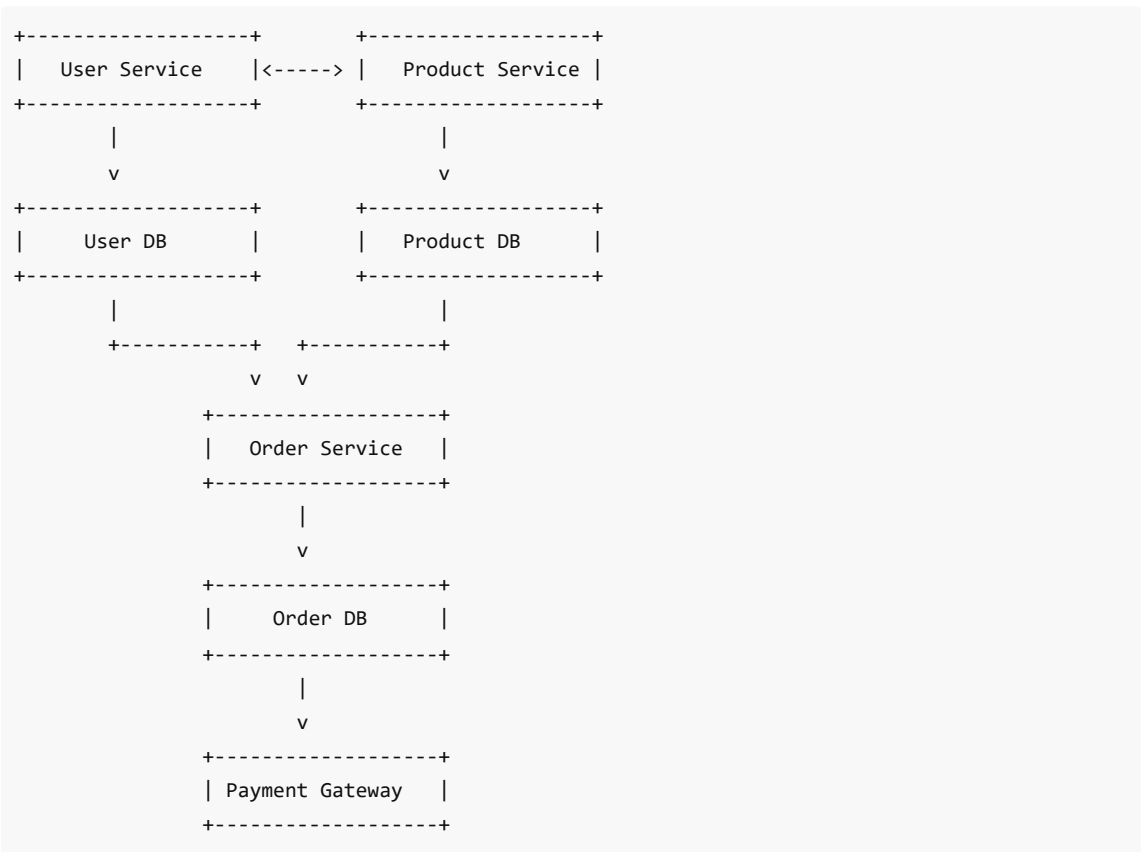
1.1 What is HLD?

- HLD shows the system's major components, their relationships, and data flow.
- Focus: Services, databases, external integrations, and communication patterns.

1.2 HLD Diagram (draw.io)

- **How to draw in draw.io:**
 - Use rectangles for services (e.g., "User Service", "Order Service").
 - Use cylinder shapes for databases (e.g., "User DB", "Product DB").
 - Use cloud shapes for external systems (e.g., "Payment Gateway").
 - Use arrows to show data flow and API calls.
 - Group related components with containers or colored backgrounds.

Example HLD (textual):



- **In draw.io:**
 - Drag shapes for each service and DB.
 - Connect with arrows (right-click → "Edit Style" for arrowheads).
 - Label each component clearly.

- Use colors to group (e.g., all DBs in blue, services in green).

2. LLD (Low-Level Design) Overview

2.1 What is LLD?

- LLD details the internal structure of each component/service.
- Focus: APIs, data models, sequence diagrams, error flows.

2.2 LLD Diagram (draw.io)

- **How to draw in draw.io:**
 - Use UML class shapes for data models (e.g., "User", "Order").
 - Use sequence diagram templates for API flows (drag from "UML" section).
 - Show REST/GraphQL endpoints as labeled arrows.
 - Add notes for error handling and retries.

Example LLD (textual):

```
User Service:
- API: POST /users, GET /users/:id
- Data Model: User { id, name, email, passwordHash }

Order Service:
- API: POST /orders, GET /orders/:id
- Data Model: Order { id, userId, productIds, total, status }

Sequence (Order Placement):
User → Order Service: POST /orders
Order Service → Product Service: GET /products/:id
Order Service → Payment Gateway: POST /pay
Order Service → Order DB: Save order
```

- **In draw.io:**
 - Use UML class shapes for each data model.
 - Use sequence diagram shapes for API call flows.
 - Connect with arrows, label each step.

3. Rules for Creating HLD/LLD Diagrams

1. **Clarity:** Every component and connection must be labeled.
 2. **Consistency:** Use the same shapes/colors for similar entities (e.g., all DBs blue).
 3. **Direction:** Arrows should clearly indicate data/request flow.
 4. **Separation:** Group related services and DBs visually.
 5. **Legend:** Add a legend for custom symbols/colors.
 6. **Scalability:** Show how new services can be added.
 7. **Error Flows:** Indicate error/retry paths with dashed arrows or notes.
 8. **External Integrations:** Use cloud shapes for third-party systems.
 9. **API Endpoints:** List key endpoints near each service.
 10. **Versioning:** Date and version your diagrams for future reference.
-

4. Demoing HLD/LLD in Interviews

- Open your draw.io diagram and walk through each component.
 - Explain why you chose each separation (e.g., "Order Service is stateless for scaling").
 - Show data flow for a key use case (e.g., order placement).
 - Point out error handling and retry logic.
 - Be ready to answer: "How would you scale X?", "What if Payment Gateway fails?"
-

5. draw.io Tips

- Use the "UML" and "Entity Relation" shape libraries.
 - Use layers to toggle between HLD and LLD.
 - Export diagrams as PNG/SVG for sharing.
 - Use comments to annotate tricky flows.
-

6. Example: E-Commerce HLD/LLD (draw.io)

- ☐ Open draw.io and create a new blank diagram.
 - ☐ Add rectangles for User, Product, Order, Payment services.
 - ☐ Add cylinders for User DB, Product DB, Order DB.
 - ☐ Add a cloud for Payment Gateway.
 - ☐ Connect with arrows as per the HLD above.
 - ☐ Add UML class shapes for User, Product, Order models.
 - ☐ Add a sequence diagram for order placement.
 - ☐ Add a legend and color code.
-

With this doc, you can confidently create, explain, and demo HLD/LLD for any real-time e-commerce app in interviews!